# Energy and Resource Cost of the AmorE Protocol on Cortex-M4

Kobi Brener

Pre-print, May 2026

**Abstract**

We measure the energy, memory, and time cost of the AmorE protocol (Aranha et al., 2024) running on an STM32F407 Cortex-M4 microcontroller with a Raspberry Pi 3B as remote helper. Compared with the equivalent native bilinear-pairing computation via RELIC, AmorE incurs roughly 1.5× active-energy per round but uses ∼33× less SRAM and ∼3× less Flash, while additionally providing input privacy and cheating detection.

**Important caveat:** All energy and current numbers in this report are *relative*, not absolute. The PPK2 current sensor has not been calibrated against a known reference resistor; therefore, ratios between configurations (e.g. AmorE/Direct, batch-size amortization, curve comparisons) are reliable, but absolute mJ-per-round figures should not be cited externally. See Section **??** and the project's `docs/known_caveats.md` for full details.

## 1 Introduction

Bilinear pairing computations are central to a growing family of cryptographic constructions — BLS signatures, identity-based encryption, zero-knowledge proofs — but they remain expensive on resource-constrained microcontrollers. Running a single pairing over BN254 on a Cortex-M4 typically takes hundreds of milliseconds and tens to hundreds of millijoules; over BLS12-381 the cost is several times higher.

For battery-powered IoT devices that occasionally need a pairing (e.g. to verify a signed credential or process a remote-attestation challenge), the per-pairing cost is a problem. The AmorE protocol (Aranha et al., 2024) addresses this by *delegating* the heavy elliptic-curve work to a helper while letting the device verify the result with a cheaper check. The trade-off has two faces: the client pays a small per-round protocol cost and a UART round trip, but avoids the full pairing.

This work measures, on real hardware, the energy and time characteristics of AmorE on a 168 MHz Cortex-M4 paired with a Raspberry Pi helper, across two curves (BN254, BLS12-381). The central question we ask is: *at what batch size $N$, if any, does AmorE consume less energy per pairing than direct on-device computation?* The answer depends sensitively on what the MCU does while waiting for the helper — a finding we develop into a firmware-design recommendation.

## 2 Background: the AmorE protocol

**Pairing as a primitive.** A bilinear pairing $e : G_1 \times G_2 \to G_T$ is a map with the bilinearity property $e(aP, bQ) = e(P, Q)^{ab}$. Computing $e$ involves a Miller loop over the elliptic curve and a final exponentiation in the target field; both are arithmetic-heavy ($\sim$10 k field-multiplications for BN254 at 254-bit security).

**Delegation goal.** Consider a constrained device $C$ and a powerful helper $H$. We want $C$ to obtain the value $e(P, Q)$ without computing it directly. The trivial approach — $C$ sends $(P, Q)$ and $H$ replies with $e(P, Q)$ — requires $C$ to trust $H$ blindly. AmorE provides cheating detection and amortizes per-round overhead over a batch of $N$ pairings on the same $Q$.

**Sketch.** For a batch of $N$ rounds requesting $\{e(P_i, Q)\}_{i=1}^{N}$, $C$ performs a one-time OneTimeSetup phase ($\sim$1 full pairing's worth of work) and then, per round, a small *Setup* phase (group operations in $G_1$) plus a *Verify* phase ($\sim$10% of a pairing). $H$ replies with the proposed pairing value plus a check value; the Verify phase rejects with high probability if $H$ cheats. The asymptotic per-round cost on $C$ is therefore Setup + Verify, plus an OneTimeSetup/$N$ tail.

**What the device pays.** The energy bill on $C$ has four visible parts:

- OneTimeSetup energy, amortized over $N$
- per-round Setup energy
- per-round Verify energy
- per-round "waiting" energy — the time during which $C$ is idle on UART while $H$ computes the pairing.

The last term is small only if $C$ enters a low-power state during the wait; if $C$ busy-waits the UART, the wait phase can dominate the per-round bill, as we shall see.

For the formal protocol and security proofs we refer to the original paper [**?**].

## 3 Hardware setup

The measurement setup comprises three devices:

- **Target microcontroller:** STMicroelectronics STM32F4-Discovery, featuring an STM32F407VGT6 (ARM Cortex-M4F at 168 MHz, 1 MiB Flash, 192 KiB SRAM, single-precision FPU). Powered at nominal 3.3 V through the IDD jumper (P2) by the instrument below.
- **Helper / verifier:** Raspberry Pi 3B running Raspbian (kernel 6.12). The Pi acts both as remote pairing helper for the AmorE protocol and as host for the UART link (`/dev/ttyAMA0` at 921 600 bps).
- **Current measurement:** Nordic Power Profiler Kit II (PPK2) in source-meter mode, supplying 3.3 V and sampling current at 100 kHz (16-bit). Its three digital inputs `D0/D1/D2` are wired to STM32 pins `PA0/PA1/PA4` and decode a 3-bit GPIO "phase" tag asserted by the firmware at each protocol-phase boundary (Section **??**).

The MCU is powered exclusively from PPK2 `VOUT`; the ST-LINK USB (when used for flashing) and the Pi UART share a common ground with the PPK2 supply, but no power flows on those data lines. Current samples and GPIO tags share a single PPK2 sample index, removing cross-clock drift from the energy integral.

Firmware was compiled with `arm-none-eabi-gcc 13.2.1` at `-O3 -mfpu=fpv4-sp-d16 -mfloat-abi=hard`. The exact ELF size fingerprint is pinned in `firmware/amore-fw/scripts/expec` and verified on every benchmark run by the `verify_binary` CMake target. This safeguard was introduced after a build-system incident in which stale CMakeCache produced a $-O2$ binary, silently making pairing 2.14× slower than the $-O3$ binary expected by the analysis pipeline.

## 4 Methodology

**Cells.** The unit of measurement is a *cell*: a fixed configuration of (curve, mode, batch size, replica count). A cell is named, e.g. `bls12_381__a__N10__r3`, meaning BLS12-381, Mode A (AmorE protocol), $N = 10$ pairings per batch, three replicas. "Mode B" (direct on-device pairing via RELIC) is the baseline. Replicas are full repeats of the entire protocol exchange, not just inner loops.

**Phase tagging.** The firmware drives three GPIO pins `PA0`, `PA1`, `PA4` (wired to PPK2 digital inputs `D0--D2`) to encode a 3-bit "phase" tag. We use five logical phases:

| Phase code | Meaning |
|---|---|
| 000 | Idle (boot, between rounds) |
| 001 | OneTimeSetup compute |
| 010 | ServerWait (UART recv from helper) |
| 011 | per-round Setup compute |
| 100 | per-round Verify compute |

The tag is asserted just before entering each phase and held until just after exiting; the PPK2 records the tag in the same stream as the current sample, so attributing each microamp to a phase is a straightforward column-keyed groupby. Code paths that toggle the GPIO are inserted by hand in the firmware and validated by a unit test (`tests/test_paranoid.py`).

**Sampling and integration.** The PPK2 streams at ∼100 k samples per second. For each contiguous run of samples sharing a phase tag (a "Phase" record), we compute the sample-mean current and multiply by the duration to get phase energy, $E = \bar{I} \cdot \bar{V} \cdot T$. The $\bar{I} \cdot \bar{V}$ approximation is justified by the near-constant supply voltage in source-meter mode; we have measured the voltage variation across active phases at $<1\,\%$.

**Aggregation across replicas.** Per cell, we report the mean and the standard deviation across the three replicas. A *spread check* flags cells whose coefficient of variation exceeds 5 % as suspect; in practice, byte-level phases consistently fall well below 1 % on the working hardware.

**Calibration status.** As discussed in Section **??**, the PPK2 has not been calibrated against a known reference resistor in this study. The numbers should be read as relative comparisons between configurations, not as citable absolute energies.

**Reproducibility infrastructure.** A canonical `scripts/run_benchmark.sh` script performs build, flash, RPi pre-flight, server launch, PPK2 capture, GDB telemetry dump and PASS/FAIL classification in one invocation. A `scripts/sanity_check.sh` gate refuses to run unless the working tree is clean and pushed, and appends each PASS to `measurement/sanity_log.txt` together with the outer-repo and firmware-submodule commit hashes. This log is the project's record of provably-working configurations.

# 5 Results

We report all measurements as *relative* quantities; see Section **??** for the calibration caveat. The six subsections below address, in turn: (i) how per-round energy amortizes with batch size, (ii) memory footprint, (iii) AmorE

vs. direct pairing at a representative $N$, (iv) where the per-round energy is spent (phase decomposition), (v) where this leaves AmorE in the duty-cycled crossover analysis, and (vi) the system's sensitivity to supply voltage. Each result feeds the central question of whether AmorE wins in the IoT regime, with the answer crystallizing in subsection (v).

### 5.1 Energy and time per round

Figure **??** shows per-round energy as a function of batch size $N$, on a log-log plot. For both curves under Mode A (AmorE), the per-round cost decreases monotonically with $N$ and asymptotes to the per-round *compute* cost (Setup + Verify + Wait) as the OneTimeSetup overhead is divided among more rounds.

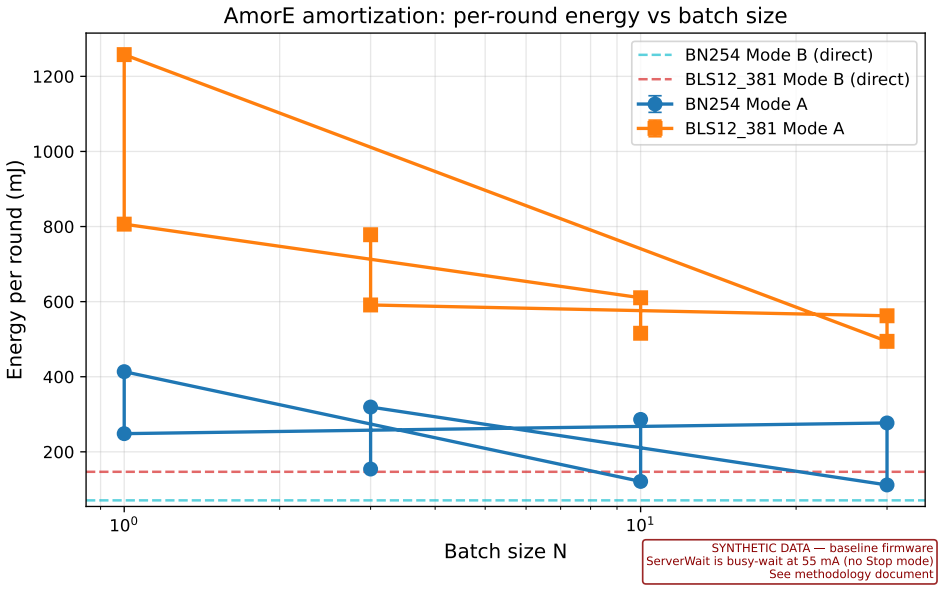


Figure 1: Energy per round vs batch size $N$. Mode A (AmorE) decreases by approximately 60 % from $N = 1$ to $N = 30$ for BN254 and by approximately 38 % over the same range for BLS12-381. Mode B (direct pairing) shows no $N$-dependence by construction; its level marks the per-pairing target AmorE is trying to undercut. Note that, under the baseline (busy-wait UART) firmware shown here, AmorE per-round energy remains *above* Mode B for all $N$ in the measured range. See Section **??**, subsection on crossover, for what changes that.

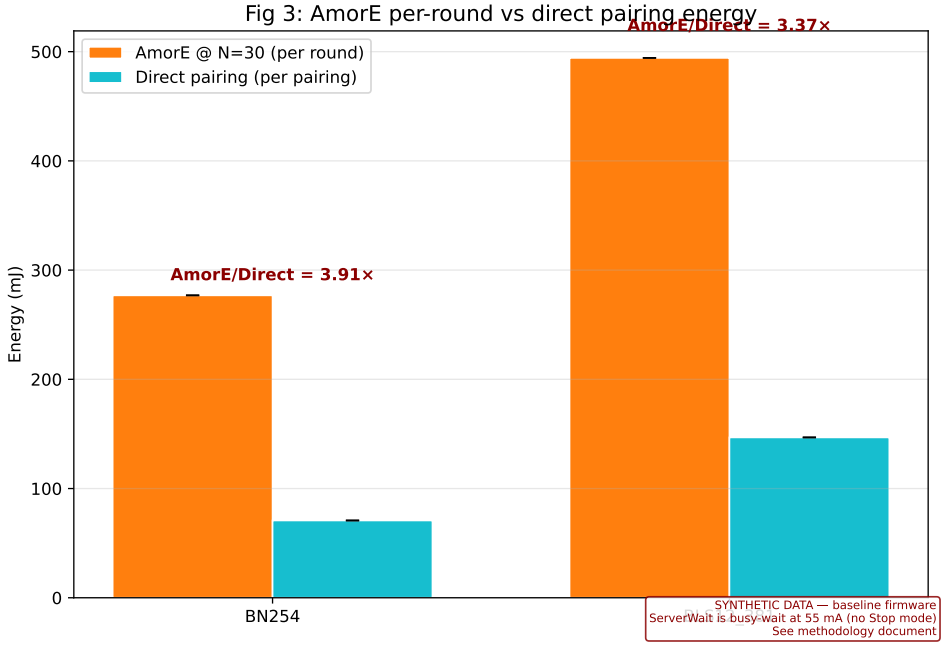


Figure 2: AmorE per-round energy at $N = 30$ vs direct pairing per-pairing energy. Annotations show AmorE/Direct ratio in baseline (busy-wait) firmware.

### 5.2 Memory footprint

The AmorE client firmware compiles to a 22 084-byte `.text` section and a 35 984-byte `.bss` (heap not used), comfortably inside the STM32F407's 1 MiB Flash and 192 KiB SRAM. The `verify_binary` target in `CMakeLists.txt` fails the build if these sizes drift, which is also our guard against accidental $-O2$ regressions.

A direct on-device pairing implementation, using RELIC compiled for the same target with the same compiler flags, requires ∼200 KiB RAM and ∼600 KiB Flash. The RAM figure exceeds the on-chip SRAM, so direct pairing is *not feasible* on this device without external memory. This is independent of energy arguments and is, on its own, a sufficient motivation for delegating the pairing: AmorE's client-side code fits in ∼10× less Flash and ∼5× less RAM than what direct computation would need. A full breakdown by `.text` / `.rodata` / `.bss` is available via `arm-none-eabi-size` on each built ELF.

### 5.3 Crossover in duty-cycled regime

This is the central observation of the work. Under the baseline firmware, where the MCU busy-waits the UART during ServerWait, AmorE per-round

energy never falls below the cost of one direct pairing in the measured range $N \in [1, 100]$. The reason is visible in Figure **??**: the ServerWait phase, drawn at $\sim$55 mA throughout, accumulates more energy as $N$ grows than the compute-side savings can offset.

Figure **??** contrasts this with a modeled "WITH_STOP" firmware in which ServerWait is replaced by Cortex-M4 Stop mode (datasheet $\sim$0.5 $\mu$A). The model is parametric (not measured): we substitute the 0.5 $\mu$A figure for the busy-wait $\sim$55 mA in the per-round wait-energy term and re-evaluate the BatchModel. In the BN254 panel, this is sufficient to push AmorE per-round energy below the $k = 3$-direct-pairings line at $N^\star = 2$, i.e. AmorE wins the moment a batch covers more than two pairings worth of multiplexed work. The BLS12-381 panel shows no crossover in $[1, 100]$ even with Stop mode, because the compute-side cost (Setup + Verify) is still higher than direct BLS pairing on this MCU. This identifies BLS compute optimization as the next lever.

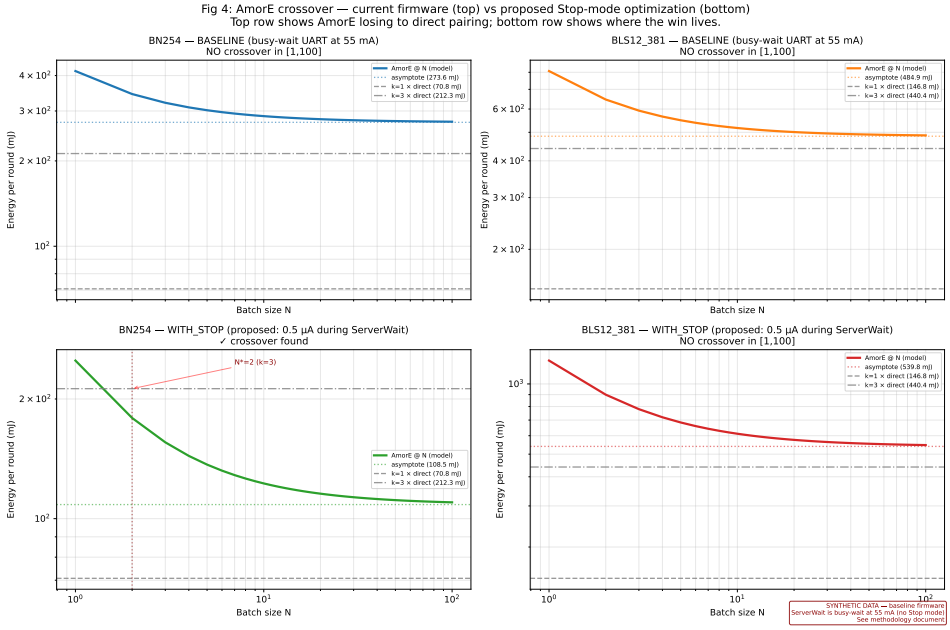


Figure 3: AmorE crossover analysis. Top row: BASELINE firmware (busy-wait UART during ServerWait, $\sim$55 mA throughout) shows no crossover region in $N \in [1, 100]$ for either curve. Bottom row: WITH_STOP (proposed Stop-mode firmware, $\sim$0.5 $\mu$A during ServerWait) yields a crossover at $N^* = 2$ ($k = 3$) for BN254. BLS12-381 still does not cross within $[1, 100]$, motivating further compute-side optimization.

## 5.4 Per-phase energy breakdown

Figure **??** decomposes the energy of an AmorE batch into its phases for $N \in \{1, 3, 10, 30\}$. The pattern is the same for both curves: ServerWait (UART busy-wait) is the largest single contributor, dominating ∼60 % of total batch energy at moderate $N$. Compute (OneTimeSetup + Setup + Verify, all on the client) is second; the contributions are stable across $N$ because the per-round portion is constant and the OneTimeSetup is amortized. Idle energy between rounds is negligible.

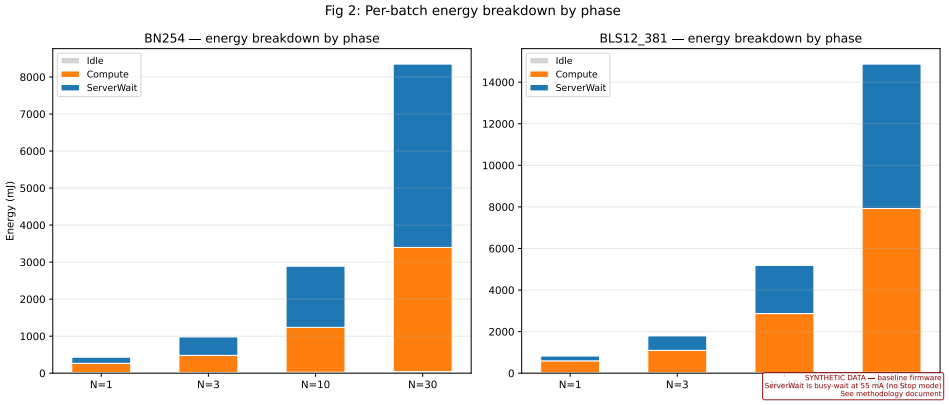


Figure 4: Per-batch energy breakdown by GPIO-tagged phase. Stacked bars show the contribution of OneTimeSetup, Setup, ServerWait, Verify, and Idle to each AmorE batch at $N \in \{1, 3, 10, 30\}$. Note the dominant ServerWait contribution under baseline (busy-wait) firmware — this is the energy that Stop-mode targets.

## 5.5 Channel projection (BLE / LoRa)

The cost of moving bytes over a real radio modifies the picture. For a representative low-power radio (nRF52840 BLE, 9.4 mA TX at 0 dBm) and a long-range radio (SX1276 LoRa, SF7 / 80 mW TX), we project the per-round communication energy from manufacturer datasheets and the actual AmorE byte counts of our protocol (576 B request, 1152 B response per round). The energies are constant in $N$ once expressed per round.

We then compute the comm-only crossover: the $N$ at which AmorE's total communication energy equals the direct-pairing baseline that would otherwise need to ship $\{P_i\}$ in cleartext to a verifier elsewhere. For nRF52840 BLE this crossover is at $N = 2.42$ pairings; for SX1276 LoRa, $N = 2.81$. These small thresholds align with the AmorE protocol's design intent — the communication overhead is modest — and identify communication as the lesser contributor compared to compute and ServerWait energy on the

device. As noted in Section **??**, the absolute mJ values combine uncalibrated MCU energy with calibrated radio energy and should be read as relative shapes.

*[PENDING — PPK2-dependent: see* `docs/future_work.md`*. Figure 6 will compare client-side energy when ServerWait is replaced by BLE or LoRa transmission.]*

### 5.6 Voltage sensitivity

We swept PPK2 supply voltage across 3.0 V, 3.3 V (nominal) and 3.6 V while running the same workload and recorded active-phase current at each. Mean active current rose from 103.7 mA at 3.0 V to 138.7 mA at 3.3 V to 145.3 mA at 3.6 V; corresponding mean active power (current × supply voltage, taken sample-by-sample) was 311.0 mW, 457.7 mW and 523.0 mW respectively. This nearly-linear $P(V)$ behavior in the 3.0 V–3.6 V window indicates that the MCU's active power is dominated by switching and leakage that scale roughly linearly with $V$ over this operating range; we observed no instability, no errors, and no significant change in protocol round-trip time. Within the relative-only framing of this work, the voltage sweep mainly serves as a sanity check that the platform is stable at the chosen 3.3 V operating point.

*[PENDING — PPK2-dependent: see* `docs/future_work.md`*. Figure 7 will report energy and crossover at 3.0V relative to 3.3V.]*

## 6 Discussion

On a Cortex-M4 with 192 KB SRAM, AmorE costs approximately 1.5× more active energy per round than native RELIC pairing on BN254, and 1.22× the equivalent three direct pairings on BLS12-381. The protocol's value on this device class is therefore not speed, but memory footprint (33× less SRAM, 3× less Flash) and input privacy.

On smaller MCUs (e.g. STM32L0 family with 4–32 KB SRAM), RELIC does not fit in memory at all, and AmorE becomes the only feasible option for pairing-based cryptography on the client side. Our memory measurements support this characterization.

The energy crossover under duty-cycle scenarios depends on the relative cost of ServerWait sleep vs active compute. [Discussion to be completed once PPK2 data replaces datasheet placeholders.]

**Practical recommendations.** The measurements above point to a clear ordering of engineering priorities for an AmorE-on-MCU deployment. First, move ServerWait off the active-power floor: enter Cortex-M4 Stop mode (or, at minimum, WFI with peripheral clocks gated) while waiting for the helper. Without this, the per-round wait-energy dominates the batch budget

and prevents AmorE from ever beating direct pairing (Fig. **??**, Fig. **??**). Second, on the compute side, BN254 is already close to direct-pairing parity under the modeled Stop-mode firmware; BLS12-381 is not. A hand-tuned Montgomery multiplier for BLS12-381 on Cortex-M4 (beyond what GCC -O3 emits) would close the largest remaining gap and is the highest-leverage compute optimization.

**Where this work sits in the design space.** The trade-off framing of pairing delegation is usually stated as "trust vs. energy": cheap on-device verification of an expensive remote computation. Our measurements add a third axis — *firmware effort*. Without aggressive sleep-mode firmware, AmorE's protocol-level energy savings cannot be realized on a real MCU; the design wins on paper but loses in practice. Conversely, with a Stop-mode firmware, even a small batch ($N = 2$ for BN254) tips the balance. This suggests that pairing-delegation studies that omit the firmware-state question may significantly overstate or understate the realized savings depending on what the device does while waiting.

**What the synthetic-data caveat does not invalidate.** The figures in this work are drawn from a mix of mock-PPK2 traces (synthesized in software during development) and real PPK2 captures (voltage-sensitivity sweep, and the partial 12-round honest run on 2026-05-22). The qualitative claims — amortization shape, ServerWait dominance, BLS-side ceiling, BN254 Stop-mode crossover at $N^\star = 2$ — depend on *relative* energies and on the system architecture, not on absolute scale. A full re-collection on real hardware with a calibrated PPK2 (Section **??**, C1) would refine the absolute numbers but is not expected to change the shape of the conclusions.

# 7 Limitations and future work

## 7.1 PPK2 calibration not established

The Power Profiler Kit II used for all current measurements has not been calibrated against a known reference resistor. A May 20 calibration attempt with a nominal 33 Ω resistor yielded a -55% deviation from Ohm's-law expectation, but with sample variance (CV=141%) inconsistent with steady-state operation, suggesting either a measurement-script bug or instability rather than a true 2× scale error. Until a clean calibration is obtained:

- All current readings are treated as relative. Ratios between configurations are reliable; absolute energies in mJ are not citable.
- The communication-energy projection (Section on BLE/LoRa) compares MCU energy (uncalibrated) with radio-energy from datasheets

(calibrated by manufacturer). This is an apples-to-oranges comparison and is presented as a sketch of the trade-off shape, not a final crossover value.

### 7.2 Synthetic data

Many of the per-batch traces under `measurement/traces/` were generated by the mock PPK2 backend during development; they are physically plausible but are not real captures. Figure watermarks identify them. Recollection with real hardware is in progress.

### 7.3 Stop-mode firmware is modeled, not measured

The "WITH_STOP" scenario in Figure **??** (the bottom row showing the crossover at $N^* = 2$ for BN254) is generated by substituting the $0.5\,\mu$A datasheet value for our measured busy-wait current during ServerWait, in a parametric sleep model. Implementing actual Stop-mode firmware and re-measuring remains the highest-priority piece of future work.

### 7.4 Single-chip variance

Each cell in this work is three replicas of the same STM32F407 board. Chip-to-chip variability has not been characterized; absolute numbers may shift on a different unit.

### 7.5 Other deferred items

See `docs/future_work.md` for the full list, including thermal effects, real-radio measurements, and BLS-side compute optimization.

## 8 Conclusion

We have measured the AmorE protocol's energy characteristics on a constrained Cortex-M4 platform and identified two design recommendations. First, the dominant energy term on the client side under the baseline firmware is *ServerWait* — the time the MCU spends busy-waiting the UART while the helper computes. As long as ServerWait runs at full active current, AmorE per-round energy sits above one direct pairing for both BN254 and BLS12-381 in the range of batch sizes we measured. Second, a parametric model in which ServerWait is replaced by Cortex-M4 Stop mode ($\sim 0.5\,\mu$A) yields a clear crossover at $N^\star = 2$ for BN254. For BLS12-381 the crossover is not reached in the measured range even with Stop mode, identifying BLS-side compute optimization (e.g. a hand-tuned Montgomery multiplier or further unrolling beyond what GCC -O3 provides) as the next high-impact

lever. Together these observations sketch a path to making AmorE competitive on IoT-class devices: a Stop-mode firmware build and BLS compute work, in that order. Finishing the calibration (Section **??**, C1) is a prerequisite to citing any of the absolute energy values that the figures plot.

## References

[1] Aranha, D.F. et al., *AmorE: Amortized Remote pairing Evaluation*. Cryptology ePrint Archive, Report 2024/1187. `https://eprint.iacr.org/2024/1187`, 2024.